

Full Implementation Guide: Proxmox HCI Cluster (Proxmox VE + Ceph + OVS + DRS)

Where We Are and Where We're Going

You have 3 physical Dell R720 servers, each with Proxmox installed, and the cluster is already formed. The architecture you're building has 3 layers:

- **Layer 0** (Physical): your 3 R720s running Proxmox bare metal
- **Layer 1** (Cluster): the 3 nodes joined as one Proxmox cluster with Ceph storage and OVS networking
- **Layer 2** (VMs): the application VMs (HAProxy, Apache x2, PostgreSQL) + infrastructure VMs (DRS service, InfluxDB+Grafana, PBS)

Your document describes a "nested virtualization" (Inception) approach where the 3 Proxmox nodes are VMs inside a single physical server. Since you have 3 **physical** servers instead, your setup is actually better and simpler since you skip the nested part entirely. Everything else applies directly.

Phase 1: Verify and Solidify the Cluster

Before touching networking or storage, confirm the cluster is healthy.

On any node:

```
pvecm status
pvecm nodes
```

Expected output should show all 3 nodes with quorum votes and no errors. If you see `Quorum information with Quorum providers: corosync_votequorum` and all 3 nodes listed, you're good to proceed.

Also check `/etc/hosts` on **all three nodes** to make sure each node can resolve the others by hostname:

```
cat /etc/hosts
```

It should have entries like:

```
192.168.20.100 pve1
192.168.20.101 pve2
192.168.20.102 pve3
```

If not, add them manually on all 3 nodes:

```
vim /etc/hosts
```

Phase 2: Network Architecture - Setting Up OVS with 9 VLANs

This is the most critical and complex phase. Your document defines 9 VLANs:

VLAN	Name	Subnet	Purpose
10	Management	192.168.10.0/24	Web UI, SSH, API
20	Corosync	192.168.20.0/24	Heartbeat, quorum
30	Ceph Cluster	10.10.30.0/24	OSD replication internal
40	Ceph Public	10.10.40.0/24	VM access to RBD storage
50	Live Migration	10.10.50.0/24	Hot migration, DRS
60	VM Production	172.16.60.0/24	Sona-Web production VMs
70	VM Test/Dev	172.16.70.0/24	Isolated test VMs
80	DMZ/External	172.16.80.0/24	HAProxy exposed endpoint
90	Backup	10.10.90.0/24	PBS backups

Step 2.1: Install Open vSwitch on All 3 Nodes

Run this on **each node**:

```
apt update
apt install -y openvswitch-switch
systemctl enable openvswitch-switch
systemctl start openvswitch-switch
```

```
# Verify it's running
ovs-vsctl show
```

Step 2.2: Plan Your Physical NICs

You have 4 NICs per server (nic0..nic3). Assign them like this before touching configs:

```
eno1 (nic0) - uplink to your physical switch (carries all VLANs via trunk)
eno2 (nic1) - bond partner for eno1 (LACP bonding for redundancy)
eno3 (nic2) - optional dedicated Ceph replication link (direct crosslinks between servers)
eno4 (nic3) - spare / iDRAC or future use
```

If your switch supports LACP (802.3ad), bond eno1+eno2 together. If not, use just eno1 as the single uplink for now. Your document mentions bond0 (LACP) feeding into the OVS bridge, so ideally:

```
bond0 (eno1+eno2, LACP) --> OVS Bridge (vbr0) --> VLAN interfaces 10,20,30...90
```

Step 2.3: Configure /etc/network/interfaces on Each Node

This is the most important config file. Back it up first:

```
cp /etc/network/interfaces /etc/network/interfaces.backup
```

Then edit it. Here is the full config for **Node 1 (pve1)**. Nodes 2 and 3 get the same structure but with their own IPs (.12 , .13):

```
vim /etc/network/interfaces

auto lo
iface lo inet loopback

# Physical NICs - manual, no IP, fed into bond
auto eno1
iface eno1 inet manual
    bond-master bond0

auto eno2
iface eno2 inet manual
```

```
bond-master bond0

# LACP Bond
auto bond0
iface bond0 inet manual
    bond-slaves eno1 eno2
    bond-miimon 100
    bond-mode 802.3ad
    bond-xmit-hash-policy layer2+3

# OVS Bridge - main bridge, bond0 is the uplink
auto vmbr0
iface vmbr0 inet manual
    ovs_type OVSBridge
    ovs_ports bond0 vlan10 vlan20 vlan30 vlan40 vlan50 vlan60 vlan70 vlan80 vlan90

# Bond as OVS port
allow-vmbr0 bond0
iface bond0 inet manual
    ovs_type OVBond
    ovs_bridge vmbr0
    ovs_bonds eno1 eno2
    ovs_options bond_mode=balance-tcp lacp=active other_config:lacp-time=fast

# VLAN 10 - Management
allow-vmbr0 vlan10
iface vlan10 inet static
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options tag=10
    address 192.168.10.11/24
    gateway 192.168.10.1

# VLAN 20 - Corosync
allow-vmbr0 vlan20
iface vlan20 inet static
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options tag=20
    address 192.168.20.11/24

# VLAN 30 - Ceph Cluster (internal OSD replication)
allow-vmbr0 vlan30
iface vlan30 inet static
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options tag=30
    address 10.10.30.11/24
```

```
# VLAN 40 - Ceph Public (VM to storage access)
allow-vmbr0 vlan40
iface vlan40 inet static
    ovs_type OVSIntPort
    ovs_bridge vmbr0
    ovs_options tag=40
    address 10.10.40.11/24

# VLAN 50 - Live Migration
allow-vmbr0 vlan50
iface vlan50 inet static
    ovs_type OVSIntPort
    ovs_bridge vmbr0
    ovs_options tag=50
    address 10.10.50.11/24

# VLAN 60 - VM Production (no IP on host, VMs use this)
allow-vmbr0 vlan60
iface vlan60 inet manual
    ovs_type OVSIntPort
    ovs_bridge vmbr0
    ovs_options tag=60

# VLAN 70 - VM Test/Dev
allow-vmbr0 vlan70
iface vlan70 inet manual
    ovs_type OVSIntPort
    ovs_bridge vmbr0
    ovs_options tag=70

# VLAN 80 - DMZ
allow-vmbr0 vlan80
iface vlan80 inet manual
    ovs_type OVSIntPort
    ovs_bridge vmbr0
    ovs_options tag=80

# VLAN 90 - Backup
allow-vmbr0 vlan90
iface vlan90 inet static
    ovs_type OVSIntPort
    ovs_bridge vmbr0
    ovs_options tag=90
    address 10.10.90.11/24
```

For Node 2, change all .11 to .12 . For Node 3, change to .13 .

Step 2.4: Apply the Network Config

Do this carefully because you WILL lose your current SSH connection briefly. Have a console/iDRAC session open as backup before running this:

```
systemctl restart networking
```

Or reboot cleanly:

```
reboot
```

After reboot, verify all VLANs are up:

```
ip addr show  
ovs-vsctl show
```

You should see all the vlan interfaces with their IPs.

Step 2.5: Update Corosync to Use VLAN 20

Now that Corosync has its own dedicated interface, update it to use `192.168.20.x` instead of whatever it was using before:

```
vim /etc/pve/corosync.conf
```

Update the `ring0_addr` in the nodelist section for each node:

```
nodelist {  
  node {  
    name: pve1  
    nodeid: 1  
    quorum_votes: 1  
    ring0_addr: 192.168.20.11  
  }  
  node {  
    name: pve2  
    nodeid: 2  
    quorum_votes: 1  
    ring0_addr: 192.168.20.12  
  }  
}
```

```
node {  
  name: pve3  
  nodeid: 3  
  quorum_votes: 1  
  ring0_addr: 192.168.20.13  
}  
}
```

Save and verify corosync picks it up:

```
pvecm status  
corosync-cfgtool -s
```

Step 2.6: Set Migration Network to VLAN 50

Tell Proxmox to use the dedicated migration VLAN for live migrations:

```
vim /etc/pve/datacenter.cfg
```

Add or update:

```
migration: secure,network=10.10.50.0/24
```

Phase 3: Ceph Distributed Storage

Your document specifies: 3 OSDs (one per node), replication factor 3, using VLAN 30 for cluster network and VLAN 40 for public network.

Step 3.1: Install Ceph on All Nodes

In the Proxmox web UI: go to your cluster node, then `Datacenter > Ceph > Install`. Select the latest stable version (Reef or Quincy). Do this on **all 3 nodes**.

Or from CLI on each node:

```
pveceph install --version reef
```

Step 3.2: Initialize Ceph on Node 1 Only

```
pveceph init --network 10.10.30.0/24 --cluster-network 10.10.30.0/24
```

This sets the Ceph cluster network. Now edit the Ceph config to also set the public network (VLAN 40):

```
vim /etc/ceph/ceph.conf
```

Under [global] add:

```
[global]
public_network = 10.10.40.0/24
cluster_network = 10.10.30.0/24
```

Step 3.3: Create Monitors on All 3 Nodes

On each node (from web UI or CLI):

```
# On pve1
pveceph mon create

# On pve2 and pve3 (SSH into each and run)
pveceph mon create
```

Verify monitors are formed and quorum is established:

```
ceph -s
ceph mon stat
```

You should see 3 mons at {...}, election epoch X, quorum 0,1,2 .

Step 3.4: Create Managers


```
# On pve1
pveceph mgr create
```

```
# On pve2 (standby)
pveceph mgr create
```

Your document shows PVE-02 as active MGR and PVE-03 as standby, which Ceph handles automatically by election.

Step 3.5: Identify Your OSD Disks

On each node, find the disk you want to use for Ceph (not the OS disk):

```
lsblk
fdisk -l
```

You're looking for a disk that is not mounted and not the OS disk. Let's say it's `/dev/sdb` on each node. Confirm it has no partitions and is completely empty:

```
wipefs -a /dev/sdb    # clear any existing signatures
```

Step 3.6: Create OSDs

On each node, create one OSD per server:

```
# On pve1
pveceph osd create /dev/sdb
```

```
# On pve2
pveceph osd create /dev/sdb
```

```
# On pve3
pveceph osd create /dev/sdb
```

After all 3 OSDs are created, check cluster status:

```
ceph -s
ceph osd tree
```

You should see 3 OSDs in `up` and `in` state, one per host in the CRUSH tree.

Step 3.7: Create the Ceph Pool for VM Disks

```
pveceph pool create vmpool --add_storages --pg_autoscale_mode on --size 3 --min_size 2
```

This creates a pool named `vmpool` with replication factor 3. Proxmox will automatically add it as a storage backend you can use for VM disks.

Verify in web UI: `Datacenter > Storage` should now show a RBD storage entry pointing to your Ceph cluster.

Verify pool health:

```
ceph osd pool ls detail
ceph df
```

Phase 4: High Availability Configuration

Step 4.1: Enable HA in the Web UI

Go to `Datacenter > HA` in the Proxmox web UI. You should see the HA Manager status. If the cluster has quorum (which it should at this point), HA is ready to be configured per VM.

Step 4.2: Verify HA Services Are Running

On any node:

```
systemctl status pve-ha-lrm    # local resource manager
systemctl status pve-ha-crm    # cluster resource manager
```

Both should be active and running.

Step 4.3: Configure Watchdog (Important for Fencing)

HA needs a way to "fence" (forcefully kill) a failed node to prevent split-brain. On each node:

```
apt install -y watchdog
```

Edit the watchdog config:

```
vim /etc/watchdog.conf
```

Add:

```
watchdog-device = /dev/watchdog
watchdog-timeout = 60
```

Enable and start:

```
systemctl enable watchdog
systemctl start watchdog
```

Then tell Proxmox to use the softdog watchdog:

```
echo "softdog" >> /etc/modules
modprobe softdog
```

Phase 5: Deploy the Application VMs (Sona-Web)

Now that storage and HA are ready, create the VMs. All VM disks go on the Ceph `vm pool` storage so they're accessible from any node (needed for HA failover).

Step 5.1: VM List to Create

VM	Role	vCPU	RAM	Disk	Network
haproxy	Load Balancer	2	2GB	16GB on vm pool	VLAN 60 + VLAN 80
apache1	Web Server 1	2	2GB	32GB on vm pool	VLAN 60
apache2	Web Server 2	2	2GB	32GB on vm pool	VLAN 60
postgresql	Database	4	8GB	100GB on vm pool	VLAN 60

Step 5.2: Create VMs via Web UI or CLI

From the web UI, when creating each VM:

- **Storage:** always select `vm_pool` (your Ceph RBD pool)
- **Network:** select `vmbr0` as the bridge and set the VLAN tag (60, 80, etc.)
- **OS:** use a Linux ISO (Debian or Ubuntu Server recommended)

Upload your ISO first: `Datacenter > local storage > ISO Images > Upload`

Step 5.3: Enable HA for Each VM

After creating the VMs, enable HA for each one:

In web UI: select the VM, go to `More > Manage HA`, set:

- **State:** started
- **Max restart:** 3
- **Max relocate:** 3

Or from CLI:

```
ha-manager add vm:101 --state started --max_restart 3
ha-manager add vm:102 --state started --max_restart 3
ha-manager add vm:103 --state started --max_restart 3
ha-manager add vm:104 --state started --max_restart 3
```

Set HA priority so HAProxy starts first, then web servers, then database:

```
ha-manager set vm:101 --group haproxy_first    # create groups with priorities in web UI
```

Step 5.4: Install Services Inside the VMs

On the HAProxy VM:

```
apt update && apt install -y haproxy

vim /etc/haproxy/haproxy.cfg
```

Add at the bottom:

```
frontend http_front
    bind *:80
    default_backend web_servers

backend web_servers
    balance roundrobin
    option httpchk GET /
    server apache1 172.16.60.11:80 check
    server apache2 172.16.60.12:80 check

systemctl enable haproxy
systemctl restart haproxy
```

On each Apache VM:

```
apt update && apt install -y apache2

# Make each server identify itself
echo "<h1>Server: $(hostname) - IP: $(hostname -I)</h1>" > /var/www/html/index.html

systemctl enable apache2
systemctl start apache2
```

On the PostgreSQL VM:

```
apt update && apt install -y postgresql

systemctl enable postgresql
systemctl start postgresql
```

Phase 6: DRS Mechanism (Dynamic Resource Scheduler)

This is the custom service your document describes, written in Python, that monitors resource usage and triggers live migrations automatically.

Step 6.1: Create the DRS + Monitoring VM

Create one more VM for the infrastructure services:

- 4 vCPU, 8GB RAM, 100GB disk on vm pool
- Connected to VLAN 10 (management, so it can reach Proxmox API and all nodes)
- Install Debian/Ubuntu

Step 6.2: Install InfluxDB on the DRS VM

```
apt update && apt install -y curl gnupg
```

```
# Add InfluxDB repo
```

```
curl https://repos.influxdata.com/influxdata-archive.key | gpg --dearmor > /usr/share/keyrings/influxdata-archive.key.gpg  
echo "deb [signed-by=/usr/share/keyrings/influxdata-archive.gpg] https://repos.influxdata.com/debian stable main" > /etc/apt/sources.list.d/influxdata.list
```

```
apt update && apt install -y influxdb2
```

```
systemctl enable influxdb  
systemctl start influxdb
```

Configure InfluxDB (access on port 8086 in browser), create:

- Organization: sonatrach
- Bucket: proxmox_metrics
- API token: save this, you'll need it

Step 6.3: Configure Proxmox Nodes to Push Metrics to InfluxDB

On **each Proxmox node**, go to web UI: Datacenter > Metric Server > Add > InfluxDB

Or via CLI on each node:

```
vim /etc/pve/status.cfg
```

Add:

```
influxdb: influxdb-metrics  
server 10.10.10.X    # IP of your DRS VM on VLAN 10  
port 8086  
organization sonatrach  
bucket proxmox_metrics  
token YOUR_INFLUXDB_TOKEN  
protocol https
```

Step 6.4: Install Grafana

```
apt install -y software-properties-common
curl https://packages.grafana.com/gpg.key | gpg --dearmor > /usr/share/keyrings/grafana.gpg
echo "deb [signed-by=/usr/share/keyrings/grafana.gpg] https://packages.grafana.com/oss.
apt update && apt install -y grafana

systemctl enable grafana-server
systemctl start grafana-server
```

Access Grafana on port 3000, connect it to InfluxDB as a data source, then import Proxmox dashboards (there are community dashboards available for Proxmox + InfluxDB).

Step 6.5: Write the DRS Python Service

```
apt install -y python3 python3-pip
pip3 install influxdb-client requests
```

Create the service file:

```
mkdir -p /opt/drs
vim /opt/drs/drs.py
```

```
import time
import requests
from influxdb_client import InfluxDBClient

# Config
PROXMOX_HOST = "192.168.10.11"    # any node, use the one with primary API
PROXMOX_USER = "root@pam"
PROXMOX_PASS = "yourpassword"
INFLUX_URL = "http://localhost:8086"
INFLUX_TOKEN = "your_influxdb_token"
INFLUX_ORG = "sonatrach"
INFLUX_BUCKET = "proxmox_metrics"

HIGH_CPU = 80          # percent
HIGH_RAM = 85          # percent
LOW_CPU = 30           # percent
LOW_RAM = 30           # percent
WINDOW = "5m"          # averaging window
```

```

COOLDOWN = 600          # seconds between migrations per VM

migrated_recently = {}

def get_proxmox_token():
    r = requests.post(
        f"https://{PROXMOX_HOST}:8006/api2/json/access/ticket",
        data={"username": PROXMOX_USER, "password": PROXMOX_PASS},
        verify=False
    )
    data = r.json()["data"]
    return data["ticket"], data["CSRFPreventionToken"]

def get_node_metrics(influx_client):
    query_api = influx_client.query_api()
    query = f'''
    from(bucket: "{INFLUX_BUCKET}")
      |> range(start: -{WINDOW})
      |> filter(fn: (r) => r["_measurement"] == "system")
      |> filter(fn: (r) => r["_field"] == "cpu" or r["_field"] == "memused")
      |> mean()
      |> group(columns: ["host", "_field"])
    '''
    return query_api.query(query, org=INFLUX_ORG)

def get_vms_on_node(ticket, node):
    r = requests.get(
        f"https://{PROXMOX_HOST}:8006/api2/json/nodes/{node}/qemu",
        cookies={"PVEAuthCookie": ticket},
        verify=False
    )
    return r.json().get("data", [])

def live_migrate(ticket, csrf, vmid, source_node, target_node):
    now = time.time()
    if vmid in migrated_recently:
        if now - migrated_recently[vmid] < COOLDOWN:
            print(f"VM {vmid} in cooldown, skipping")
            return False

    print(f"Migrating VM {vmid} from {source_node} to {target_node}")
    r = requests.post(
        f"https://{PROXMOX_HOST}:8006/api2/json/nodes/{source_node}/qemu/{vmid}/migrate",
        cookies={"PVEAuthCookie": ticket},
        headers={"CSRFPreventionToken": csrf},
        data={"target": target_node, "online": 1},
        verify=False
    )

```



```
if r.status_code == 200:
    migrated_recently[vmid] = now
    return True
return False

def main_loop():
    influx_client = InfluxDBClient(url=INFLUX_URL, token=INFLUX_TOKEN, org=INFLUX_ORG)

    while True:
        try:
            ticket, csrf = get_proxmox_token()
            metrics = get_node_metrics(influx_client)

            node_stats = {}
            for table in metrics:
                for record in table.records:
                    host = record["host"]
                    field = record["_field"]
                    value = record["_value"]
                    if host not in node_stats:
                        node_stats[host] = {}
                    node_stats[host][field] = value * 100 if field == "cpu" else value

            overloaded = []
            underloaded = []

            for node, stats in node_stats.items():
                cpu = stats.get("cpu", 0)
                if cpu > HIGH_CPU:
                    overloaded.append(node)
                elif cpu < LOW_CPU:
                    underloaded.append(node)

            for src_node in overloaded:
                if not underloaded:
                    break
                vms = get_vms_on_node(ticket, src_node)
                # sort by CPU usage descending, pick best candidate
                vms_sorted = sorted(vms, key=lambda v: v.get("cpu", 0), reverse=True)
                for vm in vms_sorted:
                    vmid = vm["vmid"]
                    target = underloaded[0]
                    success = live_migrate(ticket, csrf, vmid, src_node, target)
                    if success:
                        break

        except Exception as e:
            print(f"DRS error: {e}")
```

```
        time.sleep(60)

if __name__ == "__main__":
    main_loop()
```

Step 6.6: Run DRS as a Systemd Service

```
vim /etc/systemd/system/drs.service
```

```
[Unit]
Description=Proxmox DRS Service
After=network.target influxdb.service

[Service]
Type=simple
ExecStart=/usr/bin/python3 /opt/drs/drs.py
Restart=always
RestartSec=10
User=root

[Install]
WantedBy=multi-user.target
```

```
systemctl daemon-reload
systemctl enable drs
systemctl start drs
journalctl -fu drs    # watch logs
```

Phase 7: Proxmox Backup Server (PBS)

Step 7.1: Create PBS VM

Create one more VM:

- 2 vCPU, 4GB RAM, large disk for backup storage
- Connected to VLAN 90 (backup network)
- Download PBS ISO from proxmox.com and install it like a regular OS

Step 7.2: Configure PBS in Proxmox

After PBS is installed and running, in the Proxmox web UI:

Datacenter > Storage > Add > Proxmox Backup Server

Fill in:

- Server: IP of your PBS VM on VLAN 90
- Datastore: the backup datastore you configured in PBS
- Namespace: optional

Step 7.3: Schedule Backups

Datacenter > Backup > Add

Select all your VMs, set a schedule (daily at 2am for example), select PBS as the storage target, set retention (keep 7 daily, 4 weekly).

Summary of the Full Stack Once Done

Physical Layer:

pve1 (192.168.10.11) + pve2 (.12) + pve3 (.13)
Joined in cluster "cluster-proxmox-ve"
Corosync on VLAN 20 (192.168.20.x)

Storage Layer:

Ceph cluster: 3 MONs + 2 MGRs + 3 OSDs
Cluster net: 10.10.30.0/24 (VLAN 30)
Public net: 10.10.40.0/24 (VLAN 40)
Pool: vm_pool, replication x3, ~256GB usable

Network Layer:

OVS bridge vbr0 on each node
9 VLANs fully isolated

VM Layer:

haproxy -> VLAN 80 (DMZ) + VLAN 60 (production)
apache1 -> VLAN 60
apache2 -> VLAN 60
postgres -> VLAN 60 (disk on Ceph RBD)
drs-vm -> VLAN 10 (runs InfluxDB + Grafana + DRS Python service)
pbs-vm -> VLAN 90 (Proxmox Backup Server)

HA: all production VMs protected, auto-failover 60-120 seconds
DRS: live migration triggered at 80% CPU, cooldown 10 min
Monitoring: Grafana dashboard showing all node **and** VM metrics

The next concrete step for you right now is Phase 2 (OVS network setup) since your cluster is already formed.